Módulo 01 · Inspección del historial

Curso de Git Avanzado · Apuntes de estudio · Laboratorio: `laboratorio/m01-historial` (tienda con 3 autores: Ana, Luis, Marta)

1. Cómo referirse a commits
2. Leer el grafo: columnas, merge y padres
3. ¿Un merge crea un commit nuevo?
4. Filtrar por autor y `shortlog`
5. `log -S` : cuándo apareció/desapareció algo
6. `git show commit:ruta` y `>`
7. `--follow` : archivos renombrados
8. `log -L` : historia de una función
9. Rangos `A..B` y changelog
10. Tags y versiones (`v0.1` , `v1.0` , MVP)
11. `blame` : quién escribió cada línea
12. `--first-parent`
13. `git grep` en otra versión
14. Reto: `--name-status`
15. Chuleta final y repaso

1. Cómo referirse a commits

Sintaxis	Significado
`HEAD~3`	3 commits atrás siguiendo el primer padre
`HEAD^2`	El **segundo padre** (solo en merges)
`main@{2}`	Dónde estaba `main` hace 2 movimientos (reflog)
`v1.0.0^{commit}`	El commit al que apunta el tag
`:/texto`	El commit más reciente cuyo mensaje contiene "texto"
`A..B`	Commits en B que **no** están en A
`A...B`	Commits en A o en B, pero no en ambos
`commit:ruta`	El archivo tal como estaba en ese commit

```
git rev-parse HEAD~2       # convierte una referencia en hash
git show :/descuento       # busca un commit por su mensaje
```

2. Leer el grafo: columnas, merge y padres

```
git lg      # = log --graph --oneline --decorate --all

* 085a84f (HEAD -> main) docs: changelog v1.1
* 94b8944 fix: precio con separador de miles
* c6c86e8 feat: formatear fechas
* d221e9a refactor: limpia código sin uso
*   83a3dd3 (tag: v1.0.0) Merge branch 'feature/busqueda'
|\
| * 0b9605f (feature/busqueda) test: casos de búsqueda   ← columna derecha = la rama
| * 187a9c6 feat: búsqueda de productos
* | 87ee858 docs: documenta carrito                     ← columna izquierda = main
|/
* 1ae31ae refactor: mueve utils a src/helpers.js        ← aquí nació la rama
```

Regla: la **columna** donde está el `*` indica en qué línea se hizo el commit. El último commit de una rama es el que lleva su nombre entre paréntesis: `(HEAD -> main)` → `085a84f`.

```
git show 83a3dd3^1 --oneline -s   # 87ee858 docs: documenta carrito   (main antes del merge)
git show 83a3dd3^2 --oneline -s   # 0b9605f test: casos de búsqueda  (punta de la rama fusionada)
```

`87ee858` **fue** el último de `main` antes del merge; después `main` siguió avanzando.

3. ¿Un merge crea un commit nuevo?

Situación	¿Commit nuevo?
Las dos ramas avanzaron	✅ Sí: commit de merge con **2 padres** y hash propio
Solo avanzó la rama (main quieto)	❌ No: **fast-forward**, solo se mueve la etiqueta
`git merge --no-ff`	✅ Siempre crea el commit de merge

```
Fast-forward:
Antes:   A — B (main)            Después:  A — B — C — D (main, feature)
              \
               C — D (feature)
```

4. Filtrar por autor y `shortlog`

```
git log --author="Marta" --oneline
c6c86e8 feat: formatear fechas
0b9605f (feature/busqueda) test: casos de búsqueda
187a9c6 feat: búsqueda de productos
2e25272 feat: descuento por compra mayor a 100

git shortlog -sn --all     # -s solo conteo, -n ordenado de mayor a menor
     5  Ana
     4  Luis
     4  Marta
     1  Alumno       ← el commit de merge
```

Aparecen commits de `feature/busqueda` porque, al fusionarse, ya forman parte del historial de `main`.

5. `log -S`: cuándo apareció o desapareció algo

```
git log -S "calcularDescuento" --pretty=format:"%h %an | %s"
d221e9a Luis | refactor: limpia código sin uso          ← más reciente: lo ELIMINÓ
2e25272 Marta | feat: descuento por compra mayor a 100   ← más viejo: lo AGREGÓ

git log -S "calcularDescuento" -p --oneline   # confirma con el diff: + agregado, - eliminado
```

Opción	Busca commits donde...
`-S "texto"`	cambia el **número de apariciones** del texto (se agregó o se eliminó)
`-G "regex"`	alguna línea cambiada del diff coincide con la regex (aunque el conteo no cambie)
`--grep="texto"`	el **mensaje** del commit contiene el texto

`log` muestra primero lo más **reciente**. Con `-S`, el de abajo suele ser donde apareció y el de arriba donde desapareció. En el diff (`-p`) las líneas con `+` se agregaron y las con `-` se eliminaron.

6. `git show commit:ruta` y la redirección `>`

```
git show 2e25272:src/carrito.js     # el archivo como era en ese commit, sin checkout
export function calcularDescuento(total) {
  return total > 100 ? total * 0.1 : 0;
}

git show 2e25272:src/carrito.js > carrito-viejo.js   # lo guarda en un archivo nuevo
```

`>` no es de Git sino de la terminal: manda la salida de cualquier comando a un archivo (`git log --oneline > historial.txt`). Tu `src/carrito.js` actual no se toca; así rescatas código borrado para copiar lo que necesites.

⚠️ `>` **sobrescribe** el archivo si ya existe; `>>` agrega al final.

7. `--follow`: archivos renombrados

```
git log --oneline -- src/helpers.js           # 3 commits (solo desde que se llama así)
git log --oneline --follow -- src/helpers.js   # 4 commits
94b8944 fix: precio con separador de miles
c6c86e8 feat: formatear fechas
1ae31ae refactor: mueve utils a src/helpers.js
1f0c59b feat: utilidades de formato      ← cuando se llamaba utils.js
```

Git **no guarda** renombres: los deduce comparando el contenido. Sin `--follow`, `log -- ruta` solo mira esa ruta exacta.

8. `log -L`: historia de una función

```
git log -L :formatearPrecio:src/helpers.js
           |└──────┬──────┘ └──────┬──────┘
           |    función         archivo
           └ los dos puntos van ANTES de la función
```

Commit	Qué pasó con `formatearPrecio`
`1f0c59b`	Se **creó** (en `utils.js`, todo con `+`)
`c6c86e8`	Solo se agregó una línea vacía después
`94b8944`	`$${n.toFixed(2)}` → `Intl.NumberFormat("es-MX", {currency: "MXN"})`

`-L` = **Lines** (líneas): trabaja solo con una parte del archivo. Sigue la función incluso a través del renombre.

Formato	Qué hace
`git log -L :nombre:archivo`	historia de una función
`git log -L 1,5:archivo`	historia de un rango de líneas
`git blame -L 9,11 archivo`	autores de esas líneas (⚠️ con **espacio**, no `:`)

9. Rangos `A..B` y changelog

```
git log --oneline v1.0.0..main     # lo que tiene main y no tenía v1.0.0
085a84f (HEAD -> main) docs: changelog v1.1
94b8944 fix: precio con separador de miles
c6c86e8 feat: formatear fechas
d221e9a refactor: limpia código sin uso

git shortlog v1.0.0..main          # agrupado por autor
Ana (1):   fix: precio con separador de miles
Luis (2):  refactor: limpia código sin uso · docs: changelog v1.1
Marta (1): feat: formatear fechas

git log --pretty="- %s (%an)" v1.0.0..main   # texto listo para CHANGELOG.md
```

Un **changelog** es la lista de cambios entre una versión y otra. Se publica para que los usuarios sepan qué hay de nuevo:

```
## v1.1
- feat: formatear fechas (Marta)
- fix: precio con separador de miles (Ana)
- refactor: limpia código sin uso (Luis)
```

10. Tags y versiones

```
git tag -a v1.1.0 -m "versión 1.1"            # en el commit actual
git tag -a v0.1.0 1f0c59b -m "versión 0.1"    # en un commit viejo
git tag                                       # listar
git show v1.1.0                               # detalle
git push origin v1.1.0                        # los tags NO suben con un push normal
git push --tags                               # subir todos
```

Versionado semántico: `MAYOR.MENOR.PARCHE`

Cambio	Ejemplo	Cuándo
PARCHE	`1.0.0 → 1.0.1`	arreglaste un bug
MENOR	`1.0.0 → 1.1.0`	agregaste algo nuevo y compatible
MAYOR	`1.0.0 → 2.0.0`	cambio que rompe lo anterior
`0.x`	`0.1.0`	en desarrollo, puede cambiar todo

Flujo para publicar una versión estable

```
git switch main
git pull
git tag -a v1.0.0 -m "primera versión estable"
git push
git push origin v1.0.0
```

Momento del proyecto	Versión
Desarrollando	`v0.1.0`, `v0.2.0`...
MVP con pocos usuarios de prueba	`v0.x` o `v1.0.0-beta`
MVP en producción con usuarios reales	`v1.0.0`

⚠️ Un tag publicado **no se mueve**. Si hubo un error, sacas una versión nueva (`v1.0.1`). En GitHub los tags aparecen en **Tags / Releases**, y muchos sistemas de CI/CD se activan al subir un tag. Para volver a una versión exacta: `git switch --detach v1.0.0`.

11. `blame`: quién escribió cada línea

```
git blame src/carrito.js
1ae31ae5 (Ana   ...  1) import { formatearPrecio } from './helpers.js';
187c89a9 (Luis  ...  3) export const carrito = [];
...
909d9b9a (Luis  ...  9) export function total() {
909d9b9a (Luis  ... 10)   return carrito.reduce((s, p) => s + p.precio, 0);
909d9b9a (Luis  ... 11) }

git blame -L 9,11 src/carrito.js     # solo esas líneas
git blame -w -C -C src/helpers.js    # ignora espacios y detecta código movido/copiado
```

`blame` muestra quién tocó **por última vez** cada línea, no quién creó el archivo. La línea 1 es de Ana porque, al renombrar `utils.js`, cambió el import.

12. `--first-parent`

```
git log --oneline --first-parent main
```

Desaparecen `187a9c6` y `0b9605f`, los de `feature/busqueda`. En cada merge solo sigue el **padre 1** (`^1`, lo que ya estaba en main) e ignora el `^2`. Sirve para ver `main` como una lista limpia de "qué entró": cada feature aparece como **un solo merge**.

13. `git grep` en otra versión

```
git grep -n "export function" v1.0.0     # busca en v1.0.0 sin hacer checkout
git grep -n "export function"            # busca en la versión actual
```

Función	`v1.0.0`	Hoy
`calcularDescuento`	✅	❌ (la borró Luis)
`formatearFecha`	❌	✅ (la agregó Marta)

14. Reto: archivos por commit con `--name-status`

```
git log --author="Luis" --name-status --oneline
085a84f docs: changelog v1.1
A   CHANGELOG.md
d221e9a refactor: limpia código sin uso
M   src/carrito.js
909d9b9 feat: total del carrito
M   src/carrito.js
187c89a feat: carrito básico
A   src/carrito.js
```

`A` agregado · `M` modificado · `D` borrado · `R100` renombrado (100 % de similitud). El renombre `utils.js → helpers.js` es de **Ana**, por eso Luis no tiene `R`.

15. Chuleta final

Quiero...	Comando
Ver el grafo	`git lg`
Padres de un merge	`git show <merge>^1` / `^2`
Commits de un autor	`git log --author="X" --oneline`
Contar commits por autor	`git shortlog -sn --all`
Cuándo apareció/desapareció un texto	`git log -S "texto" -p`
Buscar en mensajes	`git log --grep="texto"`
Archivo en un commit viejo	`git show <commit>:ruta`
Historial de archivo renombrado	`git log --follow -- ruta`
Historia de una función	`git log -L :funcion:archivo`
Cambios desde un tag	`git log v1.0.0..main` / `git shortlog v1.0.0..main`
Quién escribió unas líneas	`git blame -L 9,11 archivo`
Solo la línea principal	`git log --first-parent main`
Buscar en otra versión	`git grep -n "texto" v1.0.0`
Archivos tocados por commit	`git log --name-status --oneline`
Tag más cercano	`git describe --tags`
Crear y subir un tag	`git tag -a v1.0.0 -m "..."` + `git push origin v1.0.0`

Preguntas de repaso

1. ¿Qué diferencia hay entre `log -S` y `log -G`?
2. ¿Qué diferencia hay entre `main..feature` y `main...feature`?
3. ¿Por qué hace falta `--follow` para ver el historial de un archivo renombrado?
4. En un commit de merge, ¿qué es `^1` y qué es `^2`?
5. ¿Cuándo un merge **no** crea un commit nuevo?
6. ¿Por qué `blame` puede mostrar a alguien que no creó la función?
7. ¿Qué commits oculta `--first-parent`?
8. ¿Cuándo conviene pasar de `v0.x` a `v1.0.0`?

Respuestas: (1) `-S` busca commits donde cambia el número de apariciones del texto; `-G` busca commits cuyo diff tiene líneas que coinciden con la regex, aunque el conteo no cambie. (2) `..` muestra lo que tiene `feature` y no `main`; `...` lo que tiene cada lado y el otro no. (3) Git no guarda renombres; los deduce por contenido, y `log -- ruta` solo mira esa ruta. (4) `^1` es la rama donde se hizo el merge (main antes del merge); `^2` es la punta de la rama fusionada. (5) En un fast-forward: cuando main no avanzó mientras existía la rama (salvo que uses `--no-ff`). (6) Porque muestra quién modificó **por última vez** cada línea. (7) Los commits internos de las ramas fusionadas (los que llegan por `^2`). (8) Cuando usuarios reales ya dependen del proyecto y no vas a romperles cosas sin avisar.